

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 3 — Esercitazione: una dashboard Artifact da dati sintetici anonimizzati

Anonimizzare con Claude Code

M3-E01: anonimizzare la settimana

La prima esercitazione del laboratorio, nel vostro Claude Code: sedici minuti

- Claude Code anonimizza la settimana di ticket di **CiaspoLink senza leggere il file**: scrive `anonimizza.py` dal dizionario delle colonne, lo esegue, lancia il **controllo** dei dati personali e corregge finché passa; poi `profilo.py`, gli **indicatori** della settimana
- Il file lo legge **lo script**: Claude Code manda in rete prompt e risposte, e ciò che passa da uno strumento resta su disco in chiaro
- **Percorso A**: un vostro CSV con **dati inventati** e il suo dizionario — lo stesso metodo; il controllo conosce solo i dati della settimana, quindi i vostri valori li cercate **voi** nel file anonimo, e la verifica automatica non c'è
- **Percorso B**: la settimana di CiaspoLink, così com'è — la prova del docente e la verifica sono su questo
- La guida di Claude Code: «**Give Claude a check to run** … Give Claude something it can run that comes back pass or fail.» E il ciclo che l'esercitazione mette alla prova: «It makes the change, runs the check, and if something broke it reads the output and fixes it before running the check again.»
- Il centro assistenza: «Giving Claude a way to verify its work will **markedly improve** the quality of the final result.» È un **confronto**: lo decide la coppia registrata del corso, con e senza controllo, in «Confronto», alla fine del laboratorio

Il prompt

Il prompt intero del passo 2 del README, nella forma del Modulo 2

nella sessione su Sonnet: il prompt del passo 2, 1 di 3

```
Sei uno sviluppatore Python dell'assistenza clienti di
CiaspoLink e prepari per il team di analisi una copia anonima
della settimana di ticket.

Contesto: la settimana è in dati/ticket-settimana.csv, 343
segnalazioni con dati personali, che devono restare fuori da
questa conversazione. Il dizionario delle sue colonne è
dati/README.md; gli indicatori del profilo sono in
kpi_definizioni.md.

Istruzione: scrivi ed esegui due script, in quest'ordine.
1. Leggi dati/README.md e kpi_definizioni.md.
2. Scrivi anonimizza.py, che legge dati/ticket-settimana.csv e
   scrive anonimo/ticket-anonimi.csv. Eseguilo con
   python3 anonimizza.py.
3. Scrivi profilo.py, che legge anonimo/ticket-anonimi.csv e
   scrive anonimo/profilo.json. Eseguilo con python3 profilo.py.
```

›_ **Un comando alla volta**: l'esito è l'**ultima riga** del controllo

Il prompt

Il prompt intero del passo 2 del README, nella forma del Modulo 2

nella sessione su Sonnet: il prompt del passo 2, 2 di 3

```
4. Esegui python3 controlla_pii.py: conosce i dati personali
   solo come impronte, e per ogni classe dice quanti ne restano
   e in quali segnalazioni, mai i valori; con --forme mostra la
   forma di ciascuno (lettere x, cifre 9). L'esito è nella sua
   ultima riga, «uscita: 0» oppure «uscita: 1»: finché dice 1,
   correggi anonimizza.py e riesegui anonimizza.py, profilo.py e
   il controllo.

Formato: gli script in Python 3.8 o successivo, solo libreria
standard. anonimo/ticket-anonimi.csv in UTF-8 senza BOM, fine
riga LF, con tutte le righe nell'ordine originale e queste 17
colonne in quest'ordine: id_ticket, aperto_il, chiuso_il,
canale, cliente, tipo_cliente, comune, tecnologia, categoria,
priorita, stato, oggetto, descrizione, team,
minuti_prima_risposta, minuti_risoluzione, soddisfazione.
```

›_ **Un comando alla volta**: l'esito è l'**ultima riga** del controllo

Il prompt

Il prompt intero del passo 2 del README, nella forma del Modulo 2

nella sessione su Sonnet: il prompt del passo 2, 3 di 3

```
anonimo/profilo.json con la chiave "kpi", gli indicatori di
kpi_definizioni.md, e accanto ciò che trova una profilazione del
file: campi vuoti, comuni scritti in modi diversi, segnalazioni
doppie.

Vincoli: i due CSV li aprono soltanto i tuoi script, e il loro
contenuto resta fuori da ciò che stampi. In cliente metti uno
pseudonimo al posto di id_cliente: C-001, C-002, … in ordine di
prima comparsa, lo stesso codice sempre con lo stesso
pseudonimo, vuoto se il codice è vuoto. Togli nome_cliente,
telefono, email e id_linea. In oggetto e descrizione sostituisci
con [OMESSO] ogni dato personale (telefoni, e-mail, indirizzi,
nomi di persone e di clienti, indirizzi MAC e IP, codici cliente
e di linea, utenze PPPoE, informazioni sulla salute); il resto
del testo e le altre colonne restano come sono. Esegui ogni
comando da solo, senza concatenarlo ad altri.
```

›_ **Un comando alla volta**: l'esito è l'**ultima riga** del controllo

Il controllo che Claude esegue

`controlla_pii.py`, e le impostazioni dello spazio di lavoro

Il controllo dei dati personali

- Conosce i dati personali solo come **impronte**, mai i valori: per ogni classe — nomi, telefoni, indirizzi, codici, salute, undici in tutto — dice **quanti** ne restano e in **quali segnalazioni**
- Un nome o un cognome della settimana rimasto **accanto a un [OMESSO]**, come in «Nome [OMESSO]», conta come dato rimasto
- Con `--forme`, anche la colonna e la **forma** di ciascuno: lettere x, cifre 9
- L'**ultima riga** è l'esito: «uscita: 0 (nessun dato personale rimasto)» oppure «uscita: 1 (…)»
- Ogni esecuzione aggiunge una riga a `anonimo/controlli.log`: l'ora, l'uscita, i residui

Le impostazioni dello spazio di lavoro

- In `.claude/settings.json`, `deny` di `Read` sui due CSV: ferma gli strumenti di Claude per i file e `cat`, `head`, `tail`, in **ogni modalità**
- Non ferma un programma che apre il file **da sé**, come uno script Python: così lo legge `anonimizza.py`, apposta — una **guardia**, non un muro
- `deny` di `Edit` sul controllo e sulle sue impronte: si corregge lo script, **non il controllo**
- `allow` per i tre comandi `python3` dell'esercizio; il resto in modalità **Manual**: ogni scrittura e ogni altro comando li approvate voi

Al lavoro

I passi 1–3 del README: dodici minuti, nel vostro Claude Code

```
# dalla cartella esercitazioni: il passo 1 del README
tools/prepara_esercizio.sh M3-E01
cd ~/corso-claude-code/M3-E01
claude --model sonnet --effort medium
# nella sessione: il prompt intero del passo 2 del README
```

- Il README: `m3_funzionalita_claude/s2_funzionalita_claude/m3_e2_anonimizzare/M3-E01_anonimizzare_ticket/`, nel repository delle esercitazioni
- **Checkpoint**: `anonimizza.py` scritto ed eseguito; il controllo a «**uscita: 0**»; poi `anonimo/profilo.json`
- Leggete le richieste: **approvate** la scrittura dei due script e i comandi che riconoscete (i tre `python3` dell'esercizio non chiedono niente); **No** a ogni comando che stamperebbe i due CSV, come un `python3 -c` che li apre: la regola `deny` non lo ferma
- Comandi concatenati, una classe che resta, una colonna cambiata: «Se qualcosa va storto», nel README. Chi non finisce trova una copia di riferimento del file anonimo nelle cartelle di M3-E02 e M3-E03
- L'impronta: una sessione su **Sonnet**, effort **medium**, nessun subagent, sullo stesso limite di claude.ai; controllo e verifica sono script, senza chiamate al modello. Nella registrazione del corso, il 21 ottobre 2026 (`claude -p`, senza approvazioni): il prompt in **un messaggio**, da 20 a 34 turni e da 119 a 150 secondi per sessione, il controllo eseguito **4 o 5 volte**

La verifica

Il passo 4 del README: fuori dalla sessione, uno script rifà i conti

```
/exit                       # nella sessione di Claude Code
cd -                        # di nuovo nella cartella esercitazioni
tools/verifica.sh M3-E01    # 0 fatto, 1 non ancora
```

- Ricalcola con il controllo **originale**, non con le copie dello spazio di lavoro, i dati personali rimasti, nomi a metà compresi: ne devono restare **0**. Poi il file — le 17 colonne, le **343 righe** nell'ordine, le segnalazioni per giorno, i testi con [OMESSO] e nient'altro cambiato — e i **dieci indicatori** del profilo, contro quelli veri
- Stampa, come **dato**: i dati del corso rimasti in parte (un tipo di via, un civico, il dominio di un'e-mail) e le omissioni che non coprono nessun dato; le esecuzioni registrate in `controlli.log`, **senza** un esito sul ciclo della guida: il registro lo scrive il controllo, chiunque lo esegua, e che l'abbia eseguito Claude lo dicono le chiamate della sessione
- Nella registrazione del corso (21 ottobre 2026, Claude Code 2.1.306, Sonnet, effort medium, `claude -p` senza approvazioni), tre sessioni con il controllo: **0** dati personali rimasti in tutte e tre; il controllo eseguito 4, 5 e 4 volte, l'ultima con uscita 0; la frase della guida, dalle chiamate di ogni sessione: **confermata 3 su 3**

Dare a Claude un controllo che può eseguire: lavora, controlla, corregge.